



CYNA

Supervision & logs applicatifs

Projet fil rouge — Plateforme Cyna

**Adrien CACHOUX · Romain CANER ·
Ethan MIRBEAU**

Version 1.0 — Bloc 3

Solutions de cybersécurité SaaS — SOC · EDR · XDR

Ce document décrit le **suivi des performances**, la **journalisation** et les **outils de monitoring** de la solution en production. Il répond à l'axe « Supervision & logs applicatifs » du Bloc 3.

1. Stratégie de journalisation

1.1 Niveaux et sources de logs

Source	Contenu	Emplacement
Application PHP	Exceptions inattendues (converties en 500)	<code>error_log</code> PHP → <code>stderr</code> du conteneur
Apache (conteneur web)	Accès HTTP, erreurs serveur	<code>/var/log/apache2/*.log</code> (dans le conteneur)
MySQL	Erreurs, requêtes lentes (slow query log)	Volume du conteneur <code>db</code>
Caddy (reverse proxy)	Accès HTTPS, négociation TLS, erreurs	Journal Caddy (format JSON)
Sauvegardes	Sortie du script <code>backup-db.sh</code>	<code>/var/log/cyna-backup.log</code>

Dans le code, les erreurs inattendues sont journalisées par le noyau applicatif avant d'être présentées à l'utilisateur sous forme de page 500 (voir `Cyna/src/Core/Kernel.php`, appel `error_log()`), masquant le détail technique en production (`APP_ENV=production`) et l'affichant en local.

1.2 Centralisation des logs Docker

Tous les conteneurs écrivent sur `stdout` / `stderr`, ce qui permet une collecte uniforme via le pilote de logs Docker :

```
# Suivre les logs applicatifs en direct
docker compose logs -f web

# Dernières 200 lignes de la base
docker compose logs --tail=200 db

# Filtrer les erreurs 5xx dans les logs du reverse proxy
docker compose logs caddy | grep '"status":5'
```

Pour éviter la saturation disque, la rotation est configurée dans le démon Docker (`/etc/docker/daemon.json`) :

```
{
  "log-driver": "json-file",
  "log-opts": { "max-size": "10m", "max-file": "5" }
}
```

2. Suivi des performances applicatives

2.1 Indicateurs suivis (KPI)

Indicateur	Cible (note de cadrage §4.4)	Méthode de mesure
Temps de réponse moyen (pages clés)	< 2 s	Logs Caddy (durée par requête) / tests de charge
Taux d'erreurs 5xx	< 1 % des requêtes	Comptage dans les logs
Disponibilité (uptime)	> 99 %	Sonde externe (voir §3)
Requêtes SQL lentes	0 requête > 1 s en usage nominal	slow query log MySQL
Utilisation CPU / RAM du VPS	< 80 % en pointe	<code>docker stats</code> , <code>htop</code>

2.2 Surveillance des ressources

```
# Consommation temps réel par conteneur
docker stats --no-stream

# Espace disque (important sur le palier Always Free)
df -h && docker system df
```

2.3 Détection des requêtes lentes (MySQL)

Activer le slow query log pour identifier les goulots d'étranglement :

```
SET GLOBAL slow_query_log = 'ON';
SET GLOBAL long_query_time = 1;      -- seuil : 1 seconde
SET GLOBAL slow_query_log_file = '/var/lib/mysql/slow.log';
```

Les requêtes remontées guident l'ajout d'index (cf. `idx_produits_tri` , `idx_commandes_date` du DCT §7.2).

3. Monitoring & alerting

3.1 Supervision de disponibilité (uptime)

Une **sonde externe gratuite** (UptimeRobot, Better Uptime free tier, ou une tâche `cron` maison) interroge périodiquement l'URL de santé et alerte par e-mail en cas d'indisponibilité :

```
# Vérification toutes les 5 minutes ; alerte si code != 200
*/5 * * * * curl -fsS https://cyna.example.org/ >/dev/null || \
  echo "Cyna DOWN $(date)" | mail -s "ALERTE Cyna" admin@example.org
```

3.2 Point de santé applicatif (health check)

Une route légère `GET /health` (à exposer côté application) renvoie `200 OK` et vérifie la connexion à la base, servant de cible aux sondes et au `healthcheck` Docker :

```
# extrait docker-compose (production)
healthcheck:
  test: ["CMD", "curl", "-fsS", "http://localhost/health"]
  interval: 30s
  timeout: 5s
  retries: 3
```

3.3 Évolution possible (backlog)

Pour une supervision plus riche : stack **Prometheus + Grafana** (métriques conteneurs via cAdvisor/node-exporter) ou remontée d'erreurs applicatives vers **Sentry**. Non retenu au périmètre livré (Green IT / empreinte minimale) mais documenté comme piste d'évolution.

4. Consultation des logs le jour de la démonstration

Séquence recommandée pour la soutenance (axe « Supervision & logs ») :

1. `docker compose ps` — montrer les conteneurs healthy.
2. `docker compose logs -f web` — provoquer une action (connexion, commande) et montrer la trace en direct.

3. Déclencher volontairement une erreur (URL inexistante, mauvais identifiants) et montrer la ligne de log correspondante + la page d'erreur propre.
 4. `docker stats --no-stream` — montrer la consommation maîtrisée des ressources.
-

Références

- DAT — Dossier d'Architecture Technique
- Infrastructure — sauvegardes & SSL
- Plan de tests de performance
- Gestion des incidents & bugs